

OpenCorvus: Separating Execution Ownership and Delivery Evidence in Long-Horizon Agent Work

Non-experimental working manuscript
Author names and affiliations to be supplied

10 September 2026

Abstract

Long-horizon agent work can span multiple execution attempts, tools, participants, and process failures. A completed conversation is consequently an insufficient description of what ran, who retained authority to commit a transition, and which outputs informed delivery acceptance. This paper studies these distinctions in OpenCorvus, an open-source agent runtime. We develop an analytical model that separates business identity, execution occurrences, physical ownership, and revision-specific evidence. We map the model to a fixed source snapshot, focusing on fact reduction, transaction-scoped lease checks, and artifact read-selection-publication relations. Conditional propositions characterize stale-owner rejection within guarded database transitions, evidence-reference closure, and eventual rediscovery under explicit availability and fairness assumptions. Counterexamples delimit these properties: database fencing does not establish exactly-once external effects, complete reading does not establish semantic correctness, and final delivery does not establish dependency-order compliance. We provide a prospective evaluation protocol for fault recovery, independently accepted task outcomes, and mechanism costs. *[Pending empirical evidence: comparative outcomes, effect estimates, and any claim of practical improvement await experiments.]* This manuscript reports a design analysis and protocol, not an empirical validation of the runtime.

Keywords: agent runtimes; execution identity; durable state; artifact provenance; recovery; software engineering.

1 Introduction

Large language model (LLM) agents combine generated decisions with tools and environmental feedback. Software agents make this interaction concrete: they inspect repositories, edit files, execute commands, and revise their work. SWE-agent studies interfaces for such interaction, while OpenHands provides an extensible platform for software-oriented agents [13, 16]. These capabilities support workflows in which a single user objective outlives an individual model response or process. The resulting engineering problem concerns both useful reasoning and the continuity of accountable execution.

Consider a task that asks an agent to investigate a dataset, synthesize findings, and produce a reviewed report. A synthesizer may be restarted after a tool response is lost; a reviewer may consume an early revision; a final publisher may produce a well-formed document even when the intended review has not occurred. Three questions then have different answers: did a physical operation finish, did the correct execution occurrence receive its result, and does the deliverable satisfy the requested acceptance criteria? Treating these questions as one status obscures both failures and recovery decisions.

This distinction is motivated by retained OpenCorvus project records, rather than by a new experiment. A historical seven-role workflow produced a final report while the fact checker and

report writer were created before the synthesizer reached terminal success. The retained audit also records an initial publication failure and a later publication path [10]. This case cannot establish the failure rate of the current system. It demonstrates why final output and process compliance require separate evidence. Section 9 explains the provenance and limitations of that record.

Durability itself is established territory. Temporal documents history-driven workflow progression, LangGraph persists graph checkpoints, and the OpenHands Software Agent software development kit (SDK) describes event-sourced conversation state and recovery [7, 11, 14]. Similarly, leases and causal ordering have a long history in distributed systems [3, 6]. The research question is therefore narrower than whether agents should have persistent state:

How can a runtime represent the relationship between task acceptance, exact execution occurrences, physical ownership, and evidence revisions, and which correctness properties follow from that representation under stated assumptions?

We study OpenCorvus as a concrete system at source snapshot `bd2d6bd18eda`, using source inspection and retained records. Our analytical contribution is a decomposition of this question into three inspectable boundaries: facts to control projection, ownership to guarded transitions, and observations to accepted delivery. We provide (i) an explicit semantic model, (ii) conditional propositions and counterexamples tied to selected implementation paths, and (iii) an evaluation protocol that distinguishes infrastructure continuity from task quality. These are contributions of the present manuscript. Whether the integrated design improves practical outcomes, and whether its combination warrants a stronger novelty claim, remain questions for comparison and measurement.

This scope has two consequences. First, we do not present event sourcing, leases, structured collaboration, or provenance as newly invented primitives. Second, a source-level argument is not a certification of the whole runtime. The analysis identifies implementation obligations and specifies experiments capable of falsifying stronger claims. No new model runs, fault injections, adoption studies, or comparative benchmark results are reported here.

2 Background and Related Work

Agent interfaces and collaboration. SWE-agent introduces an agent–computer interface with search, viewing, editing, and feedback facilities, and evaluates interface alternatives [16]. OpenHands combines a software execution environment with tools and delegation [13]. AutoGen organizes applications through conversable agents and conversation programming [15]. MetaGPT uses specialized roles, structured outputs, and publish–subscribe communication within standard operating procedures [5]. These studies motivate attention to interaction and handoff design. We do not infer the absence of recovery or provenance facilities in their current code from the emphasis of a particular paper version.

Runtime state and durable execution. The OpenHands Software Agent SDK is especially close prior work. Its versioned paper describes immutable agent configuration, an append-only event log, mutable conversation metadata, and reconstruction of conversations from persisted state [14]. It also separates action, execution, and observation. Any claim that OpenCorvus first makes agent execution event-based or recoverable would therefore be unsupported. AIOS studies an agent operating-system abstraction with centralized system-call scheduling and context management [8]. These mechanisms operate at different boundaries from a task acceptance decision, but their existence rules out framing resource scheduling as a new idea.

LangGraph’s version-pinned persistence documentation describes checkpoints at graph super-steps, identified by threads, with persisted state available across runs [7]. Temporal’s History

Service documentation relates history updates, mutable state, timer tasks, and transfer tasks [11]. We cite these as technical documentation, not peer-reviewed empirical results. They are direct precedents for separating durable execution state from physical work delivery. A future baseline must be selected from a specified implementation with matched capabilities; mentioning a system here does not establish experimental comparability.

Ownership and provenance. Lamport formalizes causal precedence as a partial order [6]; the presence of increasing timestamps alone does not establish the dependency relation of an application. Gray and Cheriton analyze leases with explicit failure and clock assumptions [3]. We reuse the idea of bounded authority and examine the more limited transaction boundary actually visible in OpenCorvus. The World Wide Web Consortium (W3C) PROV data model separates entities, activities, agents, and derivation relations [9]. Our artifact model draws a conceptual parallel to these distinctions; we do not claim that OpenCorvus implements the PROV standard or validates its constraints.

Evaluation of extended work. SWE-EVO constructs software evolution tasks from release histories and validates behavior using before/after execution [12]. It is a candidate workload source because evolving requirements can involve multiple changes. Its task validation does not by itself observe execution ownership or crash recovery. Consequently, our protocol separates controlled infrastructure faults from task acceptance, and treats task selection and oracle coverage as explicit validity concerns. We have not run SWE-EVO in this study.

Table 1 summarizes the role of these sources in our argument. It is a comparison of documented focus, not a feature-absence leaderboard or an exhaustive literature review. The audited versions and reading scope are recorded in the accompanying citation notes.

Source family	Documented focus used here	Implication for this study
SWE-agent; OpenHands	Tool interfaces and software execution	Hold tool access constant in comparisons.
AutoGen; MetaGPT	Conversation and role-based collaboration	Do not claim structured handoffs are new.
OpenHands SDK; AIOS	Runtime state, lifecycle, or resource handling	Compare semantic boundaries, not platform size.
LangGraph; Temporal Leases; PROV	Persisted workflow progression Bounded authority and provenance relations	Treat durable execution as prior art. State assumptions and semantic limits.
SWE-EVO	Executable software evolution tasks	Add recovery and lineage oracles separately.

Table 1: Prior work constrains the claims and evaluation design. Each row refers to the versions cited in the text.

3 Problem Formulation

3.1 Objects and research unit

We use *long-horizon work* to describe work whose acceptance depends on several causally related interactions, revisions, or execution occurrences, with possible interruption between them. Wall-clock duration, token consumption, or the number of agents alone is not a definition of difficulty. A workload should report dependency depth, revision count, tool interactions, and fault exposure separately.

Let a project be an isolation context p . A task τ is a persistent business identity within that context. An epoch e distinguishes lifecycle intervals when a task is reopened. A delivery requirement is represented by a slice revision s ; revising a requirement produces a distinct reference rather than silently changing the meaning of an earlier acceptance. A session groups conversational state, while a turn identifies a particular unit of agent execution within that session. An occurrence o identifies one execution instance under the relevant task, epoch, and dispatch identity. These are analytical categories: their concrete identifiers occupy different source structures rather than one universal tuple.

An external effect x is a tool-visible operation such as a filesystem write or a service request. Its real-world outcome may become known before, after, or never through a durable observation. An artifact reference $a = (p, \tau, i, r, h)$ denotes context, identity, revision, and payload digest. This is a normalized notation, not the literal wire format: some implementation locators carry project and task authority through their enclosing scope, and resource locators additionally include snapshot, path, media type, and length. A digest binds bytes under an integrity assumption; it is not a truth certificate.

Symbol	Meaning	Must not be inferred solely from it
τ, e	Task identity and lifecycle epoch	A particular physical execution
o	Exact execution occurrence	Every run sharing its session
ℓ	Lease for a target and owner	Permission for all external effects
a	Exact artifact reference	Correctness or relevance of its contents
q	Observation with scope and lineage	Acceptance of all requirements
d	Acceptance decision over revisions	Independent validation of that judgment

Table 2: Distinct identities support distinct forms of evidence.

3.2 State and transition model

Let $F_{\leq k}$ be a durable fact prefix under a specified consistent database view. Let C_k be configuration and policy, and L_k the lease records visible in that view. We write

$$P_{k,t} = R(F_{\leq k}, C_k, L_k, t). \quad (1)$$

Here R abstracts the relevant implementation reducers; it is not a claim that the repository contains one function with this signature. The time parameter t supports expiry and deadline classification. Durable facts and mutable ownership metadata are explicitly separate. In particular, lease expiry fields are updated by renewal and release. We do not model every runtime variable as immutable.

A hint h requests that the runtime inspect durable state. A hint is not an acceptance fact or an authority grant. A scan computes a projection and may activate eligible work, record a bounded failure, or arrange a later wake. The local driver’s revision counter coalesces requests arriving during an owned scan. A periodic enumeration of live tasks provides a second opportunity to discover durable work after a hint is lost. These mechanisms concern control progress; the LLM still chooses domain actions subject to available tools and authorization.

For a target u , define $\text{cur}(L, u)$ as the record selected by the implementation’s current-lease rule. A lease ℓ is valid for owner o at an assertion instant t when

$$V(\ell, o, u, t) \iff \ell = \text{cur}(L, u) \wedge \ell.\text{owner} = o \wedge t < \ell.\text{expiry}. \quad (2)$$

The rule includes exact lease identity. A worker’s remembered expiry does not replace a fresh assertion. The current implementation orders lease rows by activation time and identity; that ordering must be consistent with acquisition order for the intended replacement semantics.

3.3 Evidence and acceptance

Write $\text{Read}(o, a)$ when recorded read windows fully cover the bytes of the exact reference a in the required action scope. Write $\text{Select}(o, a)$ when the same scoped evidence is explicitly selected for use. For an output b , let $\text{Sources}(b)$ be its declared source references. A minimal publication condition is

$$\forall a \in \text{Sources}(b) : \quad \text{Read}(o, a) \wedge \text{Select}(o, a). \quad (3)$$

This condition is about declared references. If the declared set is empty, it holds vacuously. It does not prove that all substantively necessary sources were declared, that the producer understood them, or that b follows from them.

An acceptance decision d binds a task, its relevant lifecycle boundary, selected deliverable references, evidence references, and accepted requirement revisions. We distinguish the runtime's recorded acceptance, $A_{\text{run}}(d)$, from an independent task oracle $A_{\text{ext}}(\tau, b)$. The first establishes what the runtime recorded and attributed. The second tests whether the requested outcome is actually satisfied. The evaluation reports them separately and does not define success using the runtime decision alone.

For a declared dependency graph $G = (N, E)$, a strict dependency condition would require the relevant predecessor occurrence to complete successfully before a dependent dispatch is admitted. This is stronger than reference closure and stronger than eventually collecting all outputs. In the inspected architecture, the orchestrator is responsible for following declared workflow dependencies; the host does not universally enforce that graph as a scheduling frontier. We therefore treat dependency compliance as an observed property to evaluate, not an invariant provided by the runtime.

3.4 Failure model and assumptions

The analytical model permits process crashes, lost or repeated hints, delayed results, interrupted model streams, tool timeouts, and a worker continuing after its ownership has expired. It also permits external operations whose outcome is unknown locally. The following assumptions delimit the propositions:

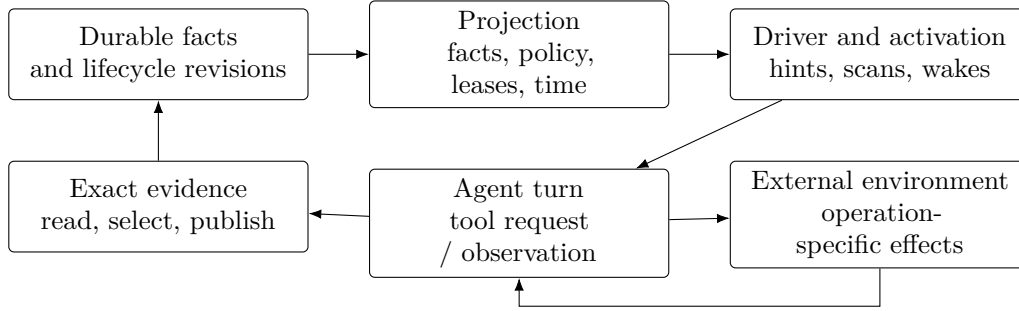
- A1. Guarded database operations use transactions with the isolation required to serialize competing ownership changes and assertions, and committed records survive the considered crashes.
 - A2. A protected transition checks the exact target, owner, and lease in the same transaction as the transition. A lease supersession requires valid domain authority. Unguarded transitions are outside this proposition.
 - A3. Lease time observations and row ordering are consistent with the intended acquisition/expiry order. Arbitrary backward clock jumps, inconsistent cross-host clocks, and storage corruption are outside the guarantee. Actual deployments must characterize these assumptions.
 - A4. Artifact storage retains the referenced version; canonical reads and digests identify its bytes correctly; the scoped read/selection collectors are faithful to actual tool outcomes. Malicious storage and digest collisions are excluded from the byte-integrity argument.
 - A5. For eventual rediscovery, the database becomes readable, the driver remains live, enumeration repeatedly includes unresolved tasks, timers eventually run, and capacity admission is fair.
- An operator-gated state additionally requires the relevant operator action.

The model provides neither Byzantine fault tolerance nor a claim that arbitrary model decisions satisfy the task. It does not assume atomic commitment across a database and an arbitrary external service. Those exclusions are essential to interpreting recovery behavior.

4 Runtime Design

4.1 Facts, projections, and physical activation

Figure 1 shows the separation used in our analysis. Domain inputs and observed outcomes contribute durable facts. Reducers interpret those facts with lifecycle policy, ownership, and time. The driver requests scans and arranges activation; a scan is not itself a new task definition. Agent turns produce real tool requests and observations, and delivery decisions refer back to identifiable evidence.



Delivery decision references evidence and accepted requirement revisions.

Figure 1: Analytical view of selected OpenCorvus paths. Arrows denote information or authority dependencies, not a claim of a single atomic pipeline. External effects cross the database boundary.

The root-ingress reducer illustrates the distinction between a pending request and permission to continue it. In the inspected source, operator abandonment and integrity faults are classified explicitly; lifecycle epochs can make older ingress inapplicable; validated decisions can resolve it; and live leases identify an already-owned activation. These alternatives matter because blindly re-running a queued instruction after a restart can apply it to a different lifecycle interval. The reducer and wake classifier expose whether progress depends on a deadline, an infrastructure fact, or operator input. A host fault is not evidence that a domain task was successfully completed.

The driver addresses a separate problem: durable work may exist without a surviving in-memory notification. When a request arrives, the local task entry’s revision increases. An existing scan owner absorbs the notification; otherwise the driver attempts capacity admission. The owner scans for a bounded number of passes, records requested wake instants, and rechecks when revisions change. Faults and non-progress are paced with backoff. The periodic live-task enumeration retains rejected admissions in its current page rather than committing past them. This mechanism supplies rediscovery opportunities without making local notification counts the business state.

The schematic procedure below is an abstraction of these selected paths. It deliberately omits policy-specific reducer branches, retry coefficients, and project entry-point wiring. It is not an executable replacement for the runtime.

1. On a hint for task τ , increment its local revision. Coalesce with a running owner or request a scan slot.
2. Under an admitted owner, read the current fact revision and invoke the scan against durable state.
3. Collect activation and wake information. If new input arrived, rescan within the pass budget; otherwise stop the pass loop.
4. On a bounded failure or lack of progress, retain the condition and arrange a later attempt under backoff.

5. Release local scan ownership and arm the earliest retained wake, if any.
6. Independently enumerate live tasks. Keep capacity-rejected entries eligible for later admission.

Local scan ownership and database activation leases serve different purposes. The former avoids redundant work within a driver instance; the latter guards selected durable transitions across physical contenders. Neither substitutes for project-scoped data access or for exact execution lineage. A complete deployment audit would need to trace every production entry point and ownership boundary; this paper does not claim to have formally verified that entire surface.

4.2 Ownership checked at the durable transition

The lease module represents a target, an owner occurrence, a lease identity, activation time, and expiry. Acquisition reads the current lease in a transaction and normally refuses a new contender while that lease is unexpired. Explicit supersession is a separate authority-bearing case. Renewal requires the current identity and owner, updates the expected expiry, and appends a grant record. Release expires the exact current lease. Thus the durable structure includes both mutable expiry and grant history.

A stale worker is rejected only where a protected transition actually rechecks ownership. For example, the dispatch-lineage commitment path accepts an admission owner, asserts its lease when present, requires a durable descriptor for the exact dispatch turn, and consumes the admission in the same transaction. The optional admission parameter is material: this code inspection does not imply that every invocation carries an admission lease. Likewise, task-completion closure uses a lifecycle-scoped target and translates failed ownership assertions into an explicit closure conflict.

Figure 2 separates database rejection from an external effect’s outcome. If a worker performs an external operation before it loses its lease, a later database rejection cannot erase that operation. If an acknowledgement is lost, the runtime may know that it lacks a result while the external service has already committed the effect. Safe recovery requires an operation-specific reconciliation or idempotency contract; the existence of a lease is insufficient.

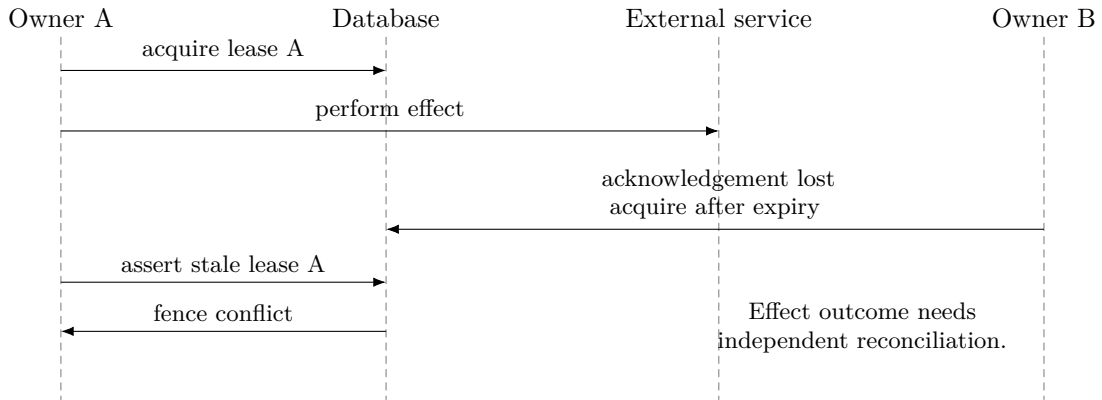


Figure 2: Schematic failure sequence, not an observed experimental trace. The stale owner’s guarded database write is rejected; whether its earlier external effect occurred requires separate evidence.

4.3 Evidence is read, selected, and published

The artifact catalog provides exact references to engine artifact revisions, task snapshots, and resources. The read path checks identity and integrity rather than interpreting an unversioned name as the latest content. Text may be read in bounded windows. The recorded windows must collectively cover the exact reference before it qualifies as a complete read in the relevant scope. A complete transport record means that the bytes were delivered through that interface; it does not reveal how a model attended to them.

The publication function accepts observed and selected locator sets from its host caller. Every declared source locator must belong to both sets. Resource checks also relate the publication to its task and validate referenced resource content. The two inspected public call paths supply these sets differently: the native tool derives scoped read and selection facts preceding the current publication, whereas the plugin host tracks reads and selections within the invocation. Both call the same catalog publication function. The authority of the input sets depends on those collectors; the function cannot prove their origin merely from a supplied array.

Figure 3 illustrates the resulting relation. Exact references prevent a reviewer from silently being credited with a later revision. Explicit selection records a producer’s declared use. Yet neither guarantees comprehensive citation: a producer can omit a relevant source or misunderstand one. Domain review and independent acceptance remain separate activities.

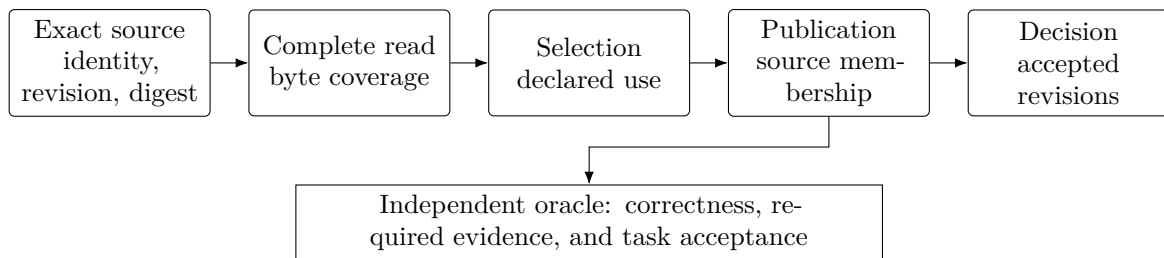


Figure 3: Evidence path for one declared source. Byte coverage, declared use, publication, and acceptance have different semantics. The independent oracle evaluates the deliverable separately.

4.4 Delivery decisions and responsibility

The inspected completion-decision schema records orchestrator session/message identity, tool call/part identity, evidence locators, deliverable locators, accepted slice revision identifiers, workflow binding, and recording time. It includes host-derived references to outputs from terminal workflow nodes. A reader locates the decision for an exact task and terminal time. This makes an acceptance decision inspectable as a particular act with referenced material.

The closure establishes a recorded set and attribution, not a proof that the model judged correctly. An output from a terminal workflow node does not certify that all preceding nodes executed in order. A passing host checker certifies only the tested contract and observed execution context. Visual or domain judgments need their own evidence; tests over unrelated code or a final status label cannot replace them. This boundary permits meaningful accounting when the runtime records completion but an independent evaluator rejects the deliverable.

5 Implementation and Audit Scope

The source inspection is fixed to commit `bd2d6bd18eda05f35326adc231289e56d0a2dfcf`. The manuscript is stored in a later documentation-only state. Product source is not modified for this paper. Table 3 provides the principal source anchors; the accompanying source map records

symbols, line spans, and pre-existing claim identifiers. The symbols are preferable to floating line numbers when a reader inspects another revision.

Module under <code>packages/opencorvus/src/</code>	Inspected responsibility
<code>engine/task-root-ingress-reducer.ts</code> <code>engine/task-control-driver.ts</code>	Classify ingress facts and wake conditions. Coalesce hints, admit scans, retain wakes, enumerate live tasks.
<code>engine/control-lease.ts</code> <code>engine/dispatch-lineage.ts</code>	Acquire, assert, renew, and release exact ownership. Bind dispatch to a durable turn descriptor; guarded admission consumption.
<code>engine/task-completion-closure.ts</code> <code>engine/completion-decision-facts.ts</code>	Lifecycle-scoped ownership of completion closure. Typed acceptance references and terminal-time lookup.
<code>artifact-catalog/index.ts</code>	Exact reads, resource integrity, source membership at publication.
<code>agent/artifact-provenance-facts.ts</code> <code>tool/artifact-catalog.ts</code> ; <code>tool/plugin-tool-host.ts</code>	Scoped complete reads and selected source facts. Supply provenance from actual tool/action scope.

Table 3: Inspected implementation anchors. This table reports source responsibilities, not runtime benchmark validation.

The repository uses TypeScript and database access through its storage abstraction; the lease acquisition wrapper uses an immediate transaction. These are implementation choices rather than prerequisites of the semantic model. The model requires an appropriate transactional boundary and persistent identity, not a particular programming language. Existing architecture documents describe capabilities and package projection, but this manuscript does not count packages, agents, providers, or tools as research contributions. Live provider projection, all production entry points, and full cross-project isolation have not been exercised for this study.

We preserve the repository’s earlier evidence package as historical input. Its machine claim matrix remains the original claim authority; the new source map relates paper passages to that package and to inspected source. It does not upgrade old implementation statuses or acceptance receipts. Source inspection can establish that a check exists in a selected path. Establishing that all actual runs traverse the intended checks requires additional execution and coverage evidence.

6 Conditional Properties and Counterexamples

The following propositions are elementary consequences of the abstract contracts and their stated assumptions. They delimit claims about the design; they are not novel distributed algorithms, machine-checked proofs, or a whole-program verification of OpenCorvus.

Proposition 1 (Stale identity at a guarded transition). *Under A1–A3, after a replacement lease becomes current for target u , a transaction asserting a different lease identity for u cannot successfully perform a transition guarded by that assertion.*

Proof. Consider the serial position of the protected transaction after the replacement transaction. A1 makes the replacement visible in its consistent view. Under A3 the current-lease rule selects the replacement. The exact-identity conjunct in Equation 2 is therefore false for the older identity. A2 places assertion and transition in the same transaction, so the failed assertion prevents that guarded transition. A transaction serialized before replacement is outside the stated temporal case. $\square$

This statement does not assert that the lease remains unexpired at physical commit time after an earlier check; it concerns the check’s serialized view. It also does not prove that unguarded

operations, arbitrary plugin actions, or external services observe the lease. The conditional guarantee must be instantiated for each protected path. A clock anomaly that changes which row is current invalidates A3 and requires a deployment-specific analysis.

Proposition 2 (Declared-source reference closure). *Assume A4 and that a publication reaches the inspected membership checks with faithful observed and selected locator sets. If the publication is accepted through those checks, every declared source has a complete read and selection in the caller’s required scope.*

Proof. Suppose an accepted publication declared a source a without a complete scoped read or without a selection. Faithful collectors would omit a from at least one supplied set. The publication loop checks membership in both sets for each declared locator and raises an error on either failure. The assumed acceptance therefore contradicts the corresponding membership check. Exact locator equality and A4 keep this argument tied to the same revision. $\square$

The proposition is intentionally limited to declared sources. Consider a report requiring evidence from documents a_1 and a_2 . A producer may read and select a_1 , omit a_2 , and publish a plausible but unsupported conclusion. Equation 3 still holds. Even declaring and reading both documents would not prove the inference valid. The oracle must therefore inspect substantive evidence coverage and reasoning separately from transport closure.

Proposition 3 (Eventual opportunity for rediscovery). *Under A5, an unresolved task that remains eligible and repeatedly appears in the live-task enumeration eventually receives an admitted scan, even if its original hint was lost.*

Proof. Repeated enumeration provides further admission requests independently of the original hint. Capacity-rejected entries remain eligible for subsequent requests. Fair admission in A5 gives an admission after finitely many such opportunities. A live driver and eventually readable storage allow the scan to occur. Without fair admission or continued inclusion in the enumeration, this conclusion does not follow. $\square$

An admitted scan is not task completion. It may correctly identify an operator-gated condition, produce another bounded failure, or dispatch an agent that makes no useful progress. Neither the proof nor the implementation inspection gives a latency bound. A bound would additionally require quantitative limits on enumeration, backoff, admission, storage response, and scan duration. This is why the evaluation measures discovery, recovery, and acceptance separately.

6.1 External effects and indistinguishable local states

Consider two executions of an external operation x . In execution H_1 , the service commits x and the acknowledgement is lost before a local observation is durable. In execution H_2 , the request never commits and the same local observation is absent. If the only available durable local facts are identical, a recovery algorithm using those facts alone cannot distinguish H_1 from H_2 . Retrying may duplicate x in H_1 ; refusing to retry may leave x undone in H_2 .

Consequently, an exactly-once claim for an arbitrary external effect requires information beyond local lease checks: for example, a service-supported idempotency key and queryable result, a shared transaction, or a domain-specific reconciliation procedure. We do not assert that OpenCorvus supplies these for every tool. The protocol records the distinct states “confirmed committed”, “confirmed not committed”, and “unknown”, and inspects the external operation independently. Local idempotent artifact publication does not resolve this general ambiguity.

6.2 Completion and causal compliance

A second counterexample separates output closure from dependency compliance. Let a workflow require $n_1 \rightarrow n_2$, where n_2 must review the accepted result of n_1 . If n_2 starts on an early artifact and n_1 later publishes a final revision, a task may eventually contain both outputs. A final decision referring to an output set does not retroactively establish that n_2 consumed the final predecessor result. The dependency property needs exact occurrence and artifact-revision relations, not only timestamps or node names.

The historical seven-role record is compatible with this distinction, but the counterexample is a logical construction and does not depend on estimating a failure probability from that record. In future execution traces, a dependency oracle must compare predecessor success, dependent dispatch, and consumed references. When cross-process order cannot be established by lineage or a common sequencer, the event order is reported as unresolved rather than inferred from unsynchronized clocks.

6.3 What these properties leave open

The propositions do not establish global linearizability, arbitrary-process isolation, complete application-level idempotency, or semantic task success. Their practical value is to expose obligations that can be checked: transaction placement, exact identity, scoped read collectors, live-task coverage, and external effect oracles. If an obligation fails, an implementation-level claim must be narrowed or the implementation investigated. The analysis does not reinterpret a failing run as a success because another layer happened to produce a deliverable.

7 Prospective Evaluation Methodology

This section specifies a study to be executed after the non-experimental manuscript. It is not a completed preregistration or a report of experiments. The parameter choices below are prospective design choices, not measured values. Task eligibility, baseline revisions, provider identity, and pilot-based sample size must be frozen in a dated execution manifest before confirmatory runs. No such runs have been performed for this manuscript.

7.1 Research questions and units

RQ1: Under defined faults, do exact execution and ownership relations recover to the prescribed outcome, and at what delay? The experimental unit is a task–fault scenario; repeated attempts are nested within that unit. The recovery oracle examines facts, exact occurrences, and independently observed effects, rather than process availability alone.

RQ2: At a fixed model and total resource budget, does OpenCorvus change independently accepted delivery relative to a matched runtime? The unit is an eligible task, with repeated runs per condition. The primary estimand is the task-weighted difference in acceptance probability on the frozen eligible task set. This is not a population claim about arbitrary user work.

RQ3: Which inspected mechanisms contribute to observed behavior, and what overhead accompanies them? The unit and pairing follow RQ1 or RQ2 according to the endpoint. Mechanism-specific ablations should be evaluated against the same frozen full system, not against an independently tuned prompt.

7.2 Workloads and eligibility

We propose two workload families. Controlled fault scenarios exercise real processes, persistent storage, and an isolated external operation with an independent ledger. They include uninterrupted controls and targeted interruptions. Software tasks assess accepted delivery under the same model/tool conditions. SWE-EVO is a candidate source [12]; no task from it has yet been accepted into this study’s execution manifest.

Before observing condition outcomes, a task is eligible only if its source and test versions can be retained, its license permits the planned use, its initial state can be reconstructed, and an acceptance oracle can check its stated requirements. Environment installation failures during this pre-screening are recorded as exclusions with reasons. After enrollment, environment or infrastructure failures are outcomes, not retrospectively removable cases. Tasks with previously public maintainer solutions, including the repository’s Actual demonstration, are excluded from the primary evaluation. A separately labeled qualitative analysis may discuss them.

We stratify eligible tasks using properties available before runs: repository, dependency depth where explicitly specified, requirement count, and expected interaction categories. We do not stratify on observed completion time or successful agent trajectories. Public benchmark tasks may be represented in model training; a fixed model does not remove that threat. Report the provenance and exposure of every task, and limit interpretations accordingly.

All infrastructure scenarios cover serial and concurrent activity, live and terminal task states, a reopened epoch, and at least two independent project contexts. Shared mechanisms must be checked across their production callers before a local scenario can support a general recovery claim. This coverage requirement is an obligation of the future experiment, not a coverage result of this manuscript.

7.3 Conditions and fairness

The intended comparison has three kinds of condition. A minimal single-agent tool loop establishes a simple reference point. A production runtime with durable state, preferably a pinned OpenHands SDK implementation or another technically compatible system, supplies the principal external comparison. The fixed OpenCorvus snapshot supplies the treatment. The external baseline is chosen and documented before task outcomes are seen, based on support for the prescribed model and equivalent tool capabilities. If no compatible baseline exists, report that limitation and restrict the claims; do not substitute published scores from a different model.

Each condition receives the same task input, repository snapshot, permitted tool operations, network scope, acceptance oracle, and aggregate resource budget. System-specific interface adapters are documented because exact prompt equivalence can be counterproductive when tool schemas differ. Give every condition the same predeclared development opportunity on disjoint pilot tasks. Freeze prompts and adapters before confirmatory execution. Report native differences, including concurrency and persistence, rather than silently removing them to improve a score.

Proposed ablations isolate (a) periodic rediscovery while retaining direct hints, (b) the selected transaction-scoped ownership checks in a disposable laboratory build, and (c) explicit evidence selection while retaining exact-version reads. These are experimental interventions, not production changes or supported compatibility paths. The ownership ablation must use isolated operations whose duplicate effects are observable and harmless. For each intervention, enumerate affected call sites, prove that the intended mechanism actually changed through a manipulation check, and document any secondary semantic change. If clean separation is impossible, analyze the intervention as a combined change rather than attribute its effect to one mechanism.

7.4 Model, budget, and execution manifest

The intended model is `openai/gpt-5.6-luna`. This is the requested catalog identity; an execution manifest must also record the actual provider and request model identifier, interface version, catalog revision, non-secret configuration digest, source commit, and package digest. The experiment may proceed only after confirming that the provider request uses the intended model. This statement does not claim that the manuscript-authoring session used that model.

For planning, use a per-run cap of 100,000 total input-plus-output tokens, 100 model requests, and 60 minutes of agent execution, whichever is reached first. These are proposed resource caps, not cost estimates or claims of adequate performance. Aggregate usage across all workers and recovery attempts; count cached input tokens in the token cap and report their billed category separately. Streaming requests are counted on initiation, including failed requests, with provider-reported token usage where available. A run with unknown token usage cannot be certified as budget-compliant and is flagged separately. Up to four simultaneously active workers may be used where supported; serialize operations that require exclusive mutable resources. Report actual concurrency.

The planned confirmatory design uses three fresh runs per task and condition, with condition order randomized within task/repetition blocks. Environment setup and final external evaluation are timed and reported separately from the agent cap. Host downtime from an injected fault remains inside elapsed-time outcomes. No manual debugging is allowed during a primary run. A required human decision is recorded as an intervention and the unassisted endpoint is evaluated at that boundary. Assisted continuations form a separate analysis and do not replace the primary attempt.

The final values may change after a disjoint pilot if the initial caps make the study uninformative or infeasible. Such changes require a dated rationale and a new manifest before confirmatory runs; pilot outcomes are then excluded from the confirmatory estimates. This is a prospective planning rule, not permission to tune budgets after seeing confirmatory results. No model calls or payments are authorized or initiated by this manuscript.

7.5 Fault injection and outcome oracles

Faults are triggered at observable boundaries identified by exact event or transaction markers. A scheduled random wall-clock kill is insufficient unless its actual boundary is subsequently established. Table 4 defines the planned scenarios. Fault duration, trigger implementation, affected process, and restart procedure belong in each scenario manifest.

For each enrolled scenario, the oracle specification lists the expected persistent fact set, allowed state transitions, exact occurrence relations, allowed external effect count, and expected terminal or operator-gated outcome. A surfaced ambiguous external result may be the correct safety outcome for F4; it is not scored as completed useful work. Record separately: duplicate physical executions, duplicate durable settlements, duplicate external effects, and outcomes that remain unknown. Their denominators differ and must not be collapsed into one error counter.

Use a monotonic observer clock for duration within one observation host. To infer cross-process precedence, use durable lineage or a common event sequencer. If either is unavailable, classify the ordering as unresolved. Recovery latency begins at restoration of the failed resource and ends when the expected durable outcome is observed; additionally report total disruption time from fault onset. Timeouts are failures for bounded recovery success and right-censored observations for descriptive recovery-time analysis. Report the restricted mean time up to the declared timeout if a time summary is used; do not summarize only successful recoveries.

The independent task oracle combines version-pinned executable checks with a requirement-level rubric prepared before execution. Hidden checks must remain inaccessible to agents. Two

Scenario	Trigger and perturbation	Required oracle evidence
F1	After durable ingress, before local hint; drop hint and restart isolated driver.	Exact ingress retained; later scan discovers it; permitted outcome recorded.
F2	After lease acquisition, before operation; stop owner and permit expiry.	Replacement identity and occurrence are explicit; old owner cannot commit a guarded transition.
F3	After child descriptor is durable, before dispatch admission consumption; interrupt owner.	Descriptor, dispatch identity, admission state, and resulting settlement agree.
F4	After an isolated external effect commits, before its observation; lose response.	Independent effect ledger distinguishes commit, duplication, and unknown local knowledge.
F5	During a streamed response, before a complete durable outcome; break connection.	Partial output is classified; an unrelated/new occurrence is not credited with its result.
F6	After a task closes or reopens, before delivery of an old hint/result.	Exact lifecycle boundary is honored; stale ingress receives its defined inapplicable/error outcome.
F7	Concurrent tasks in two projects with saturated scan capacity.	Project-scoped ownership and fair admission observations; rejected work remains discoverable.

Table 4: Planned fault scenarios. Every row requires a real storage/process path and an independent observer; this table contains no executed outcomes.

reviewers, blinded to condition when feasible, assess criteria that need domain judgment. They record initial ratings before adjudication by a third reviewer. Report raw agreement and an appropriate chance-corrected agreement statistic with its limitations. Automated LLM judgments may assist review but cannot be the sole correctness authority. User-interface outcomes require real interaction and human visual inspection under repository policy, not automated UI assertions.

7.6 Endpoints and statistical analysis

Define $Y_{icr} = 1$ only when task i , condition c , repetition r meets all preregistered mandatory acceptance criteria within budget, without manual rescue. Otherwise $Y_{icr} = 0$. A runtime completion label alone cannot set $Y = 1$. Let R_i be the planned repetitions and N the eligible task count. The primary contrast against the frozen principal baseline b is

$$\hat{\Delta} = \frac{1}{N} \sum_{i=1}^N \left(\frac{1}{R_i} \sum_{r=1}^{R_i} Y_{i,OC,r} - \frac{1}{R_i} \sum_{r=1}^{R_i} Y_{i,b,r} \right). \quad (4)$$

Equal task weighting prevents tasks with more observed retries from receiving additional weight. Planned repetitions, including failures, remain in the denominator. Infrastructure failure, timeout, missing deliverable, and unknown budget compliance count as zero for the strict primary endpoint and receive separate reason codes. Missing evaluator data require worst/best-case bounds as a sensitivity analysis, not silent exclusion or a favorable imputation. Partial satisfaction is a secondary ordinal rubric outcome.

The primary interval conditions on the enrolled tasks. Let p_{ic} be task i 's acceptance probability under frozen condition c , so that $\Delta = N^{-1} \sum_i (p_{i,OC} - p_{ib})$. Assume repetitions within each task-condition cell are independent Bernoulli trials with a stable probability. For the $2N$ cells in the principal comparison, construct equal-tailed exact binomial intervals $[L_{ic}, U_{ic}]$ by the method of Clopper and Pearson [1], each with error probability $\delta = 0.05/(2N)$. With $K_{ic} = \sum_r Y_{icr}$, use beta quantiles $L_{ic} = Q_{\delta/2}(K_{ic}, R_i - K_{ic} + 1)$ and $U_{ic} = Q_{1-\delta/2}(K_{ic} + 1, R_i - K_{ic})$; set $L_{ic} = 0$ when $K_{ic} = 0$ and $U_{ic} = 1$ when $K_{ic} = R_i$. Here $Q_q(a, b)$ is the q quantile of a beta distribution

with parameters a, b .

$$I_{\Delta} = \left[\frac{1}{N} \sum_i (L_{i,OC} - U_{ib}), \quad \frac{1}{N} \sum_i (U_{i,OC} - L_{ib}) \right]. \quad (5)$$

The union bound gives simultaneous coverage of all $2N$ cell probabilities of at least 95%; their inclusion implies $\Delta \in I_{\Delta}$. This construction needs no independence between cells and accommodates paired conditions, although it does not exploit pairing to shorten the interval. It is deliberately conservative and may be very wide with three repetitions. Report the point estimate and this interval in percentage points, with the cell counts and assumptions. If temporal provider drift or dependent repetitions invalidate the binomial model, report descriptive estimates and labeled sensitivity analyses without claiming nominal coverage.

As a separate task-heterogeneity sensitivity analysis, apply task-cluster resampling following the bootstrap principle [2]: resample whole tasks 10,000 times, retaining their paired conditions and repetitions, and report the 2.5th and 97.5th percentile range. This range is not the primary fixed-task probability interval. Also report repository-stratified estimates and leave-one-repository-out contrasts. Neither analysis establishes generalization from the curated set to arbitrary user work, and repetitions are never relabeled as independent tasks.

RQ1 reports correctly classified recovery outcomes, invariant violations by defined opportunity, unresolved effects, and timing distributions for every fault type. Report numerators and denominators explicitly. A zero observed violation count is accompanied by an upper uncertainty bound under a stated sampling model; it is not a proof of safety. RQ3 reports paired changes in acceptance and recovery, aggregate tokens and calls, storage/scan work, elapsed time, and human intervention. Costs are reported only from actual provider usage and contemporaneous billing data, with currency/date and cached-token treatment.

There is one primary comparison in Equation 4. Secondary confirmatory acceptance comparisons of the three ablations against the full system, if retained, form one family of three and use Holm-adjusted p -values at family-wise level 0.05 [4]. For each, compute a two-sided paired randomization test by swapping full-system and ablation labels jointly across all repetitions within each task. Enumerate all task-level swaps when feasible, otherwise use 10,000 random swaps and the plus-one Monte Carlo correction. This test requires the corresponding exchangeability null; it is not justified merely by repeated runs. Recovery-only interventions and cost endpoints remain descriptive. Effect sizes and intervals remain central; a non-significant result does not establish equivalence. Any additional baseline, subgroup, or interaction analysis is labeled exploratory. No post-hoc choice of the best repetition or successful subset enters the primary endpoint.

Sample size planning must use Equation 5, not a task-bootstrap variance. First freeze an eligible task set without observing condition outcomes, then evaluate candidate repetition counts using the exact binomial interval widths over all possible cell counts. Use the worst-case combined half-width to assess the prespecified five-percentage-point precision target; a disjoint pilot may additionally provide a labeled expected-width projection, but cannot replace the conservative calculation. Three repetitions are the initial planning setting, not a claim that the precision target is feasible. Increasing only the number of tasks does not remove wide cell-wise uncertainty in this conservative construction. If the authorized budget cannot support sufficient repetitions, freeze a feasible count before confirmatory execution and report the achievable precision and unmet target. Do not append runs until a favorable interval or significance threshold appears.

8 Results: Reserved for Empirical Evidence

[Pending empirical evidence: No confirmatory or pilot experiment has been run for this manuscript.] The final results section will answer RQ1–RQ3 using the protocol above. Table 5 is

a reporting specification, not a numeric result table. It contains no zeroes that could be mistaken for measured failure-free performance.

Question	Required result fields	Current state
RQ1	Scenario opportunities; correct outcomes; duplicate/unknown effects; recovery intervals	Not run
RQ2	Eligible tasks; planned runs; accepted counts; paired difference and uncertainty	Not run
RQ3	Intervention fidelity; paired effects; tokens, time, storage, human effort	Not run
Failures	All reason codes; representative trace IDs; excluded/pilot manifests	No study traces

Table 5: Reserved reporting fields. Historical demonstrations and website constants are not substituted for these missing observations.

[Pending empirical evidence: The abstract’s empirical sentence, the interpretation of comparative effects, and the final empirical conclusion must be written after these results exist.]

9 Discussion and Threats to Validity

9.1 What source evidence establishes

The analysis demonstrates that selected source paths expose distinctions that can be inspected and tested. It does not show that all deployments exercise them correctly. Database configuration, timer behavior, clock discipline, adapter fidelity, and callers that omit optional ownership arguments can alter practical guarantees. A full safety audit would need to enumerate and verify every relevant transition, including cancellation, restart, terminal cleanup, and project isolation. No formal refinement proof connects the entire implementation to Equation 1.

Provenance checks protect a narrower property than substantive correctness. A source can be faithfully transported and still be false, irrelevant, or misunderstood. Selection can omit necessary evidence. A schema-valid completion decision may embody an incorrect acceptance judgment. This is why evidence transport, dependency compliance, and independent task quality are different evaluation dimensions. More logging alone would not resolve these distinctions.

9.2 Historical records and selection bias

The retained seven-role case is a historical process record. Its evidence ledger records overlap between analysts and creation of downstream roles before synthesizer success. The foundation audit documents that the fact checker considered early material and that an initial typed publication path did not complete before a later publication path produced the report. The underlying historical database is not part of the retained artifact, so exact locator replay cannot be independently repeated from this package alone. We use the record as motivation for a failure mode, not as current-version failure evidence or as an evaluated workflow success.

Other project materials contain website benchmark constants without the complete versioned dataset, evaluator, per-case outputs, or repeated-run records needed for scientific interpretation. Those numbers do not appear in our result table or effect calculations. A maintainer-assisted Actual case is likewise a selected demonstration with intervention and repair history, not independent adoption evidence. The existence of a final report, patch, or public issue does not turn these materials into a random sample.

Historical foundation acceptance receipts describe their own stage and review state. They are not a certification of this manuscript. We preserve those receipts rather than rewriting them to imply present scientific validation. Any future reproduction must retain raw failures as well as eventual successes.

9.3 Internal, construct, and external validity

Interface adaptation is a principal internal-validity risk. A baseline might receive a weaker tool schema, less favorable context handling, or less tuning. Equal nominal token caps do not ensure equal usable context. We therefore require visible adapters, disjoint development tasks, aggregate accounting, and intervention records. Calendar-time model updates and provider nondeterminism remain possible even with a fixed catalog name; request identity and observation date are necessary replication information.

The acceptance oracle is a construct-validity risk. Passing tests may miss requested behavior, while a rubric may reward presentation rather than correctness. A recovery status may describe availability rather than safe continuation. Our separate endpoint definitions reduce these conflationations but do not eliminate incomplete or biased evaluation. Reviewer blinding may be imperfect because output formats reveal the runtime. Initial ratings, disagreement, adjudication, and hidden-test coverage should be disclosed.

External validity is limited by a single proposed model, selected software workloads, and the specific infrastructure configuration. Repository clustering and public-task contamination further restrict generalization. None of these experiments, even if successful, would by itself establish broad community adoption, economic value, self-improvement across tasks, or correctness of arbitrary autonomous agents. Those claims need separate evidence and study designs.

9.4 Operational implications and open questions

The design suggests a practical diagnostic order: identify the business revision being evaluated, identify the exact execution occurrence, inspect current ownership at the transition, and then examine the artifact versions actually consumed. This is a procedure for locating evidence, not an automated guarantee of diagnosis. Where external outcomes are unknown, the runtime must expose uncertainty to an authorized decision maker rather than convert it into a fabricated success.

Several questions remain open. Do explicit reference and selection steps impose enough interaction cost to offset their diagnostic value? Do periodic scans remain responsive under heavy multi-project contention? Which kinds of task benefit from separating recorded acceptance from an independent oracle? The proposed ablations and fault scenarios address parts of these questions; only their results can support a practical effectiveness claim.

10 Conclusion

We have presented a source-grounded analysis of OpenCorvus that distinguishes business acceptance, execution occurrences, physical ownership, and revision-specific evidence. Under explicit assumptions, selected mechanisms support stale-identity rejection at guarded database transitions, closure over declared source references, and eventual opportunities to rediscover eligible work. Counterexamples establish the limits of these statements: local durability does not imply exactly-once external effects, reference closure does not imply correct reasoning, and final delivery does not imply dependency-order compliance.

The accompanying protocol defines how to evaluate recovery, independently accepted outcomes, and mechanism costs without conflating them. *[Pending empirical evidence: Whether*

these mechanisms improve outcomes relative to a matched runtime, and at what cost, remains unanswered until the proposed experiments are performed.] This is a complete non-experimental working argument, with empirical sections explicitly reserved. It is not a submission-ready empirical paper.

References

- [1] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404. URL <https://doi.org/10.1093/biomet/26.4.404>.
- [2] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1): 1–26, 1979. URL https://blogs.helsinki.fi/bk-club/files/2012/05/Efron_Bootstrap_AS1979.pdf.
- [3] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles*, pages 202–210, 1989. URL <https://www.cs.cmu.edu/afs/cs.cmu.edu/academic/class/15712-s12/www/papers/gray89.pdf>.
- [4] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL <https://www.ime.usp.br/~abe/lista/pdf4R8xPVzCnX.pdf>.
- [5] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiaowu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. arXiv:2308.00352, version 7, 2023. URL <https://arxiv.org/abs/2308.00352v7>.
- [6] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978. URL <https://www.microsoft.com/en-us/research/publication/time-clocks-ordering-events-distributed-system/>.
- [7] LangChain contributors. LangGraph persistence documentation. Technical documentation, source revision a86eb4c5d0a8, 2025. URL <https://github.com/langchain-ai/langgraph/blob/a86eb4c5d0a82e52f63d7f83d25713052766fe84/docs/docs/concepts/persistence.md>. Accessed 10 September 2026.
- [8] Kai Mei, Xi Zhu, Wujiang Xu, Wenye Hua, Mingyu Jin, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. AIOS: LLM agent operating system. arXiv:2403.16971, version 4, 2024. URL <https://arxiv.org/abs/2403.16971v4>.
- [9] Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium, 2013. URL <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>. Edited by Moreau and Missier; 30 April 2013.
- [10] OpenCorvus contributors. OpenCorvus: Source snapshot and retained project evidence. Software and project records, revision bd2d6bd18eda, 2026. URL <https://github.com/yangheng95/opencorvus/tree/bd2d6bd18eda05f35326adc231289e56d0a2dfcf>. Source inspection; not an independently reproduced experiment.
- [11] Temporal contributors. History service architecture. Technical documentation, source revision 25d19f4f8f41, 2026. URL <https://github.com/temporalio/temporal/blob/25d19f4f8f413d5da929225259f457cbad122200/docs/architecture/history-service.md>. Accessed 10 September 2026.
- [12] Minh V. T. Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. arXiv:2512.18470, version 1, 2025. URL <https://arxiv.org/abs/2512.18470v1>.
- [13] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan,

- Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. arXiv:2407.16741, version 3, 2024. URL <https://arxiv.org/abs/2407.16741v3>.
- [14] Xingyao Wang, Simon Rosenberg, Juan Michelini, Calvin Smith, Hoang Tran, Engel Nyst, Rohit Malhotra, Xuhui Zhou, Valerie Chen, Robert Brennan, and Graham Neubig. The OpenHands software agent SDK: A composable and extensible foundation for production agents. arXiv:2511.03690, version 2, 2025. URL <https://arxiv.org/abs/2511.03690v2>. Version 2 revised in 2026; cited version is the arXiv manuscript.
- [15] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, version 2, 2023. URL <https://arxiv.org/abs/2308.08155v2>.
- [16] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv:2405.15793, version 3, 2024. URL <https://arxiv.org/abs/2405.15793v3>.

A Artifact, Sources, and Reproduction

A.1 Source snapshot and evidence map

The analyzed source snapshot is available in the public repository [10]. The manuscript artifact contains LaTeX source, BibTeX references, editable TikZ figures, citation verification notes, and a passage-to-source map. The map references the pre-existing claim matrix rather than creating a second implementation-status registry. It also distinguishes source facts from definitions, conditional deductions, hypothetical counterexamples, historical observations, and proposed experiments.

The earlier foundation directories for evidence, cases, experiments, figure inputs, and integration are unchanged by manuscript authoring. Their presence supports auditability of retained material, not full historical replay. Future experimental traces require a separate execution manifest and preserved environment; rebuilding this document does not run the system or reproduce a benchmark.

A.2 Document build

From the manuscript directory, build the single source tree with:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
```

The document uses pdfLaTeX, BibTeX, and standard TeX packages. TikZ figure sources are included directly in the manuscript. The build must resolve citations and cross-references. Inspect page rendering, embedded fonts, table/figure boundaries, and extraction order before delivery. Document checks are not evidence of runtime effectiveness.

A.3 Future execution record

Each run record must identify the task, condition, repetition, randomized order, source commit, environment image or dependency lock digest, model request identity, prompt and package digests, tool permissions, start/end observer times, budget consumption, manual interventions, and raw trace locations. Fault records add the exact boundary marker, affected process, fault duration, recovery procedure, expected facts, observed facts, and external effect ledger. Preserve a hash manifest over the non-secret artifacts.

The acceptance record must include the oracle version, checked requirement revisions, executable-check outputs, blinded initial ratings, adjudicated result, and failure reasons. Analysis scripts must implement the frozen estimand and denominator rules. Publish unsuccessful and missing outcomes alongside accepted ones. Credentials, private user data, and access tokens do not belong in prompts, traces, figures, manifests, or the repository.

A.4 Authorship and disclosure

Author names, affiliations, contribution statements, and a target venue have not been supplied. They must be supplied by the actual authors; no affiliation or human contribution is inferred. This working manuscript was drafted with an AI assistant and requires accountable author review of its sources, arguments, and eventual results. The final disclosure should describe the actual assistance and comply with the selected venue’s policy at submission time. A model name in the future experimental protocol does not identify the model used for every authoring step.

No new agent experiment, payment, external effect benchmark, user study, or paper submission was performed as part of document creation. An experimental result can only be added after the corresponding execution and evaluation artifacts exist.